Data and Artificial Intelligence

Cyber Shujaa Program

Week 9 Assignment
Classification Models in Python
Student Name: Arnold Aijuka
Student ID: CS-DA03-26044

Table Of Contents

Table Of Contents..... 2

Introduction..... 3

Tasks Completed..... 3

Load the Dataset using fetch_california_housing..... 3

Apply preprocessing using ColumnTransformer..... 4

Split the Data into training and testing sets..... 5

Use a KNeighborsRegressor..... 6

Apply GridSearchCV..... 8

Define an evaluation metric R2..... 9

Save the trained pipeline using pickle or joblib..... 9

[OPTIONAL] Using Streamlit to host model for deployment..... 10

Link to Code: Arnold_Aijuka_MLOPs_CS-DA03-26044.ipynb..... 11

Link to Streamlit App: Streamlit KNNRegressor App..... 11

Conclusion..... 11

Introduction

In this assignment, you will build an end-to-end machine learning workflow to predict housing prices using the California Housing dataset. The focus is on applying key concepts covered in class: preprocessing, cross-validation, hyperparameter tuning, pipeline construction, and model deployment.

The California Housing dataset is a well-known regression dataset that contains information collected from the 1990 California census. It is available directly through scikit-learn.

Target Variable: Median house value (in $100,000s)
Features (8):

- MedInc: median income in block group
- HouseAge: median house age in block group
- AveRooms: average number of rooms per household
- AveBedrms: average number of bedrooms per household
- Population: block group population
- AveOccup: average number of household members
- Latitude: block group latitude
- Longitude: block group longitude

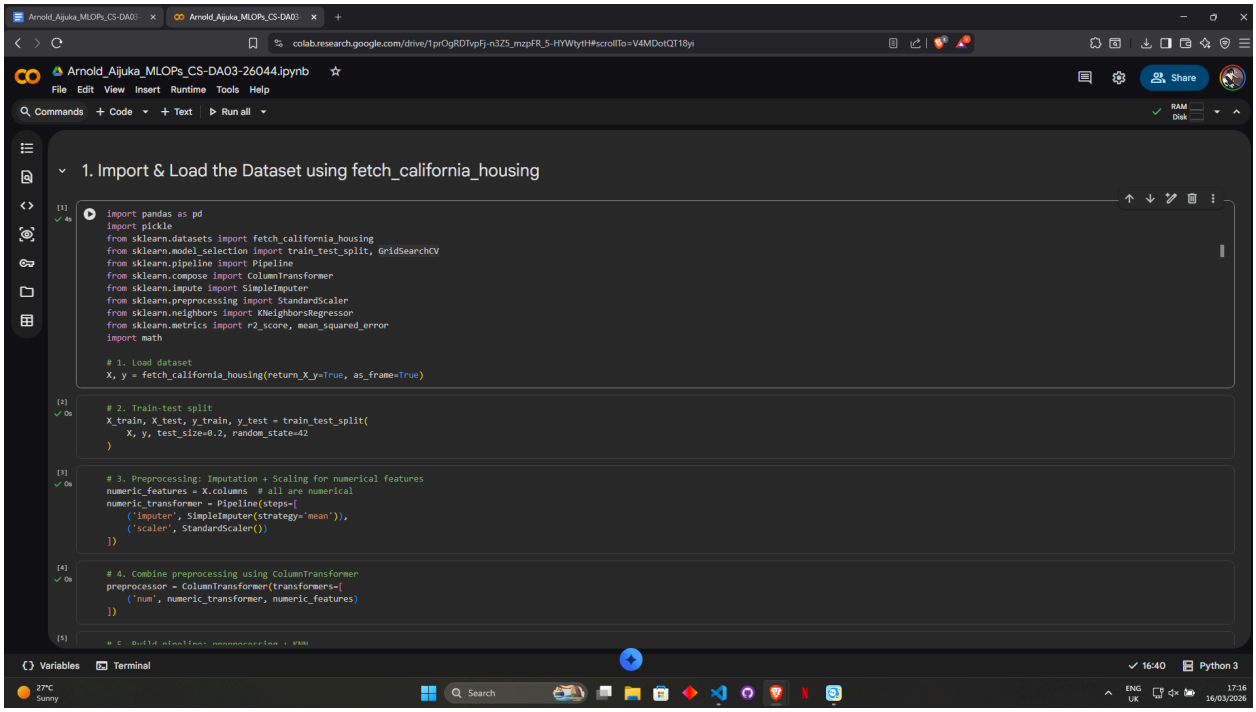
Complete the following tasks:

1. Load the dataset using fetch_california_housing.
2. Apply preprocessing using ColumnTransformer
3. Split the data into training and test sets (80/20 split).
4. Define an evaluation metric: **R^2 Score.**
5. Use a KNeighborsRegressor.
6. Apply GridSearchCV to tune the following hyperparameters: n_neighbors: [3, 5, 7, 9], weights: ['uniform', 'distance'], p: [1, 2]
7. Use **5-fold cross-validation.**
8. Save the trained pipeline using pickle or joblib.
9. **(OPTIONAL)**
    a. Create a simple **Flask or FastAPI** app with a /predict endpoint.
    b. Load the .pkl model and return predictions for incoming JSON input.

Tasks Completed

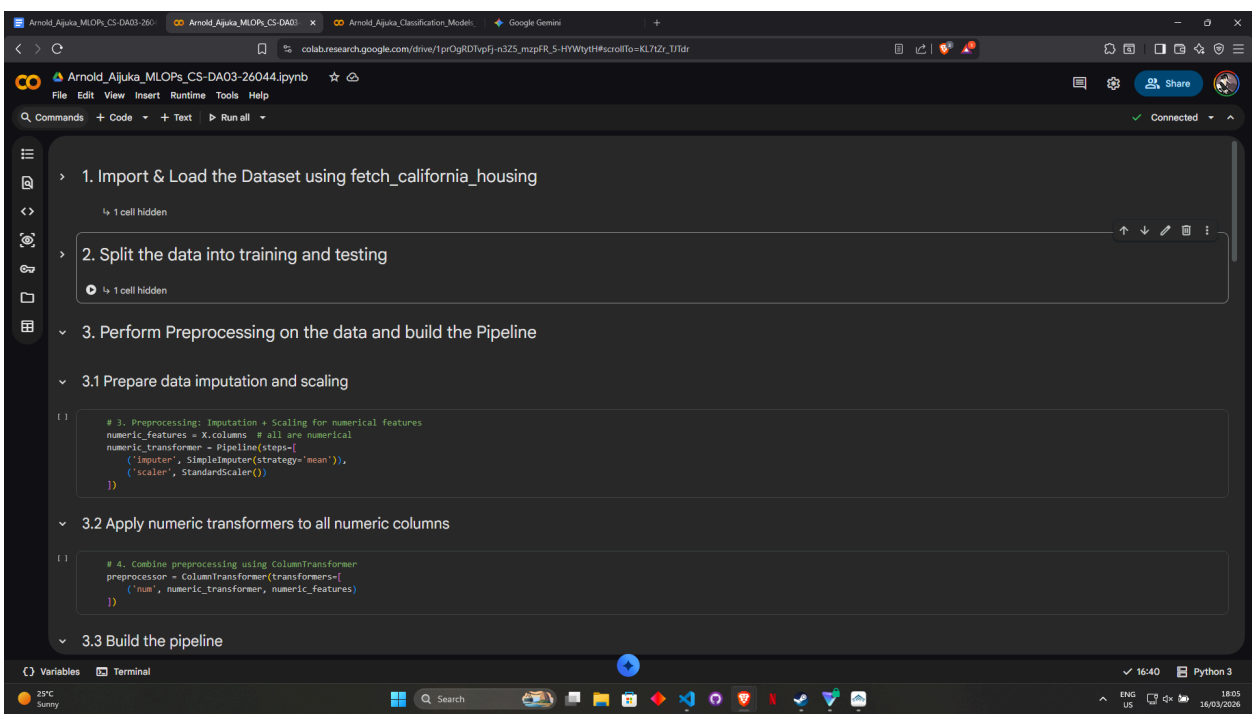
Load the Dataset using fetch_california_housing

The first step is to load the necessary libraries including the California_housing dataset and ensure the dataset is split into X and y values.

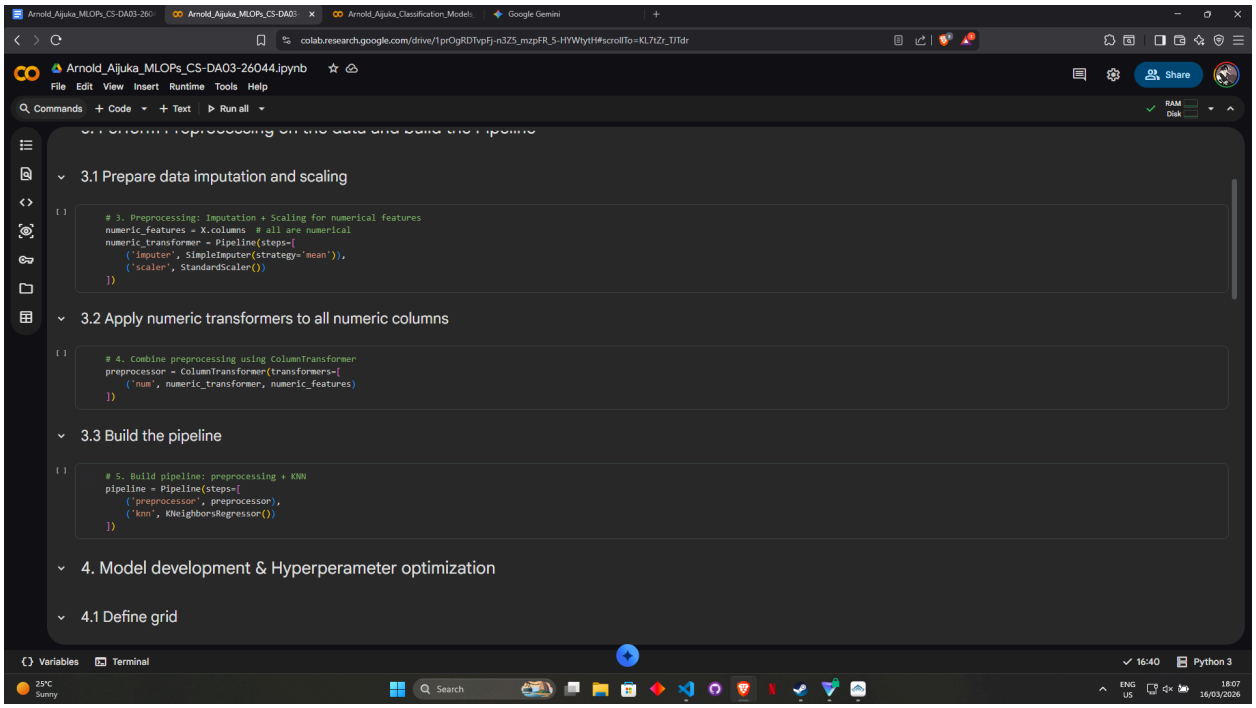


Apply preprocessing using ColumnTransformer

After loading the dataset we must then check all numerical columns (Which luckily for us is every column) and impute them with a scaler and simple imputer. Once that is defined we can then properly apply the column transfer which will apply all numeric transformers to all the numeric columns in the dataset.

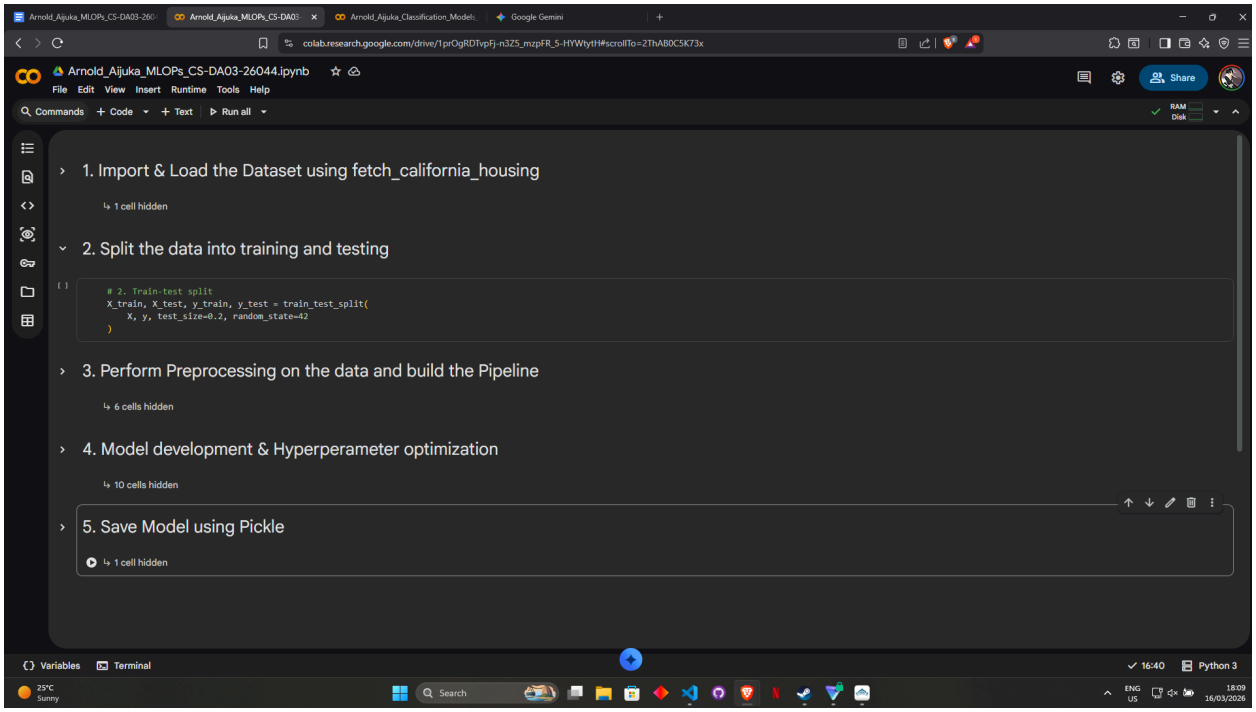


After doing that the next step is to build the pipeline and ensure it runs the preprocessor first before the model.



Split the Data into training and testing sets

We can make use of the sklearn **train_test_split** to split the dataset in an 80/20 ratio, 80 for training and 20 for testing, we will have 4 new variables **X_train, X_test, y_train, y_test**.



Use a KNeighborsRegressor.

The pipeline is the first instance where we state the model type that will be used in our GridSearchCV hyperparameter optimization in order to find the best possible parameters for our model. We use the defined grid parameters stated in the instructions earlier.

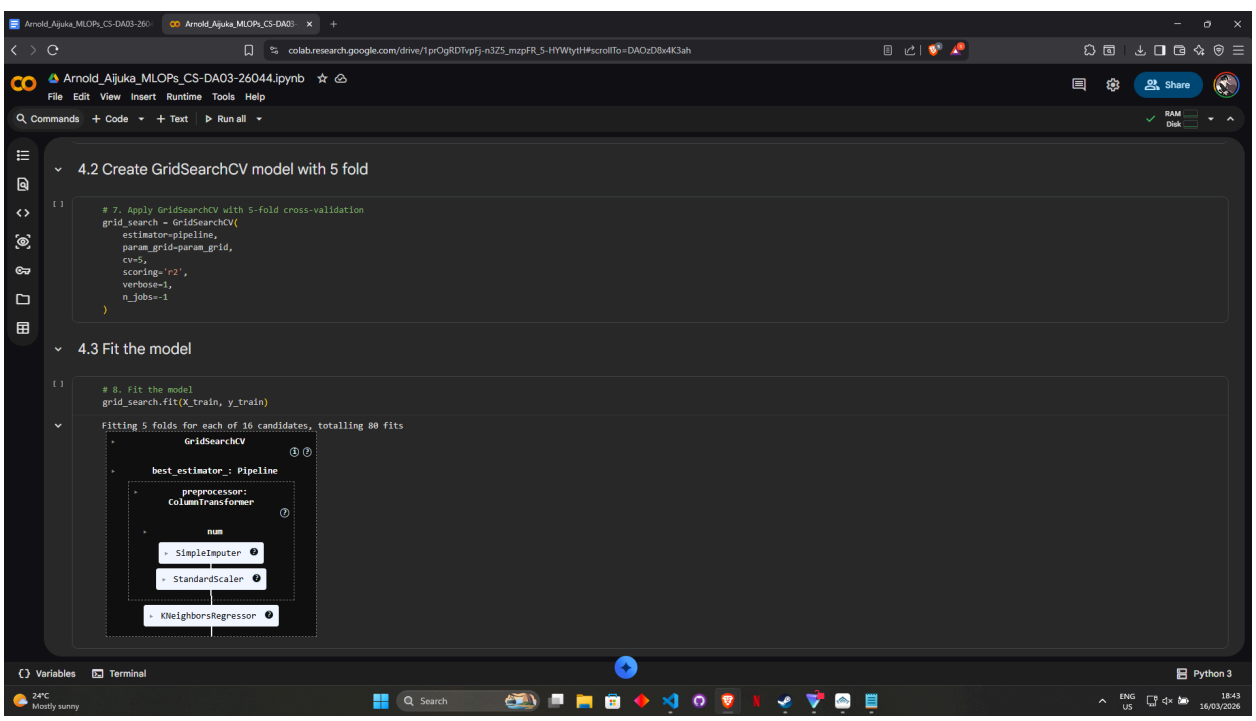
3.3 Build the pipeline

```python
# 5. Build pipeline: preprocessing + KNN
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('knn', KNeighborsRegressor())
])
```

4. Model development & Hyperperameter optimization

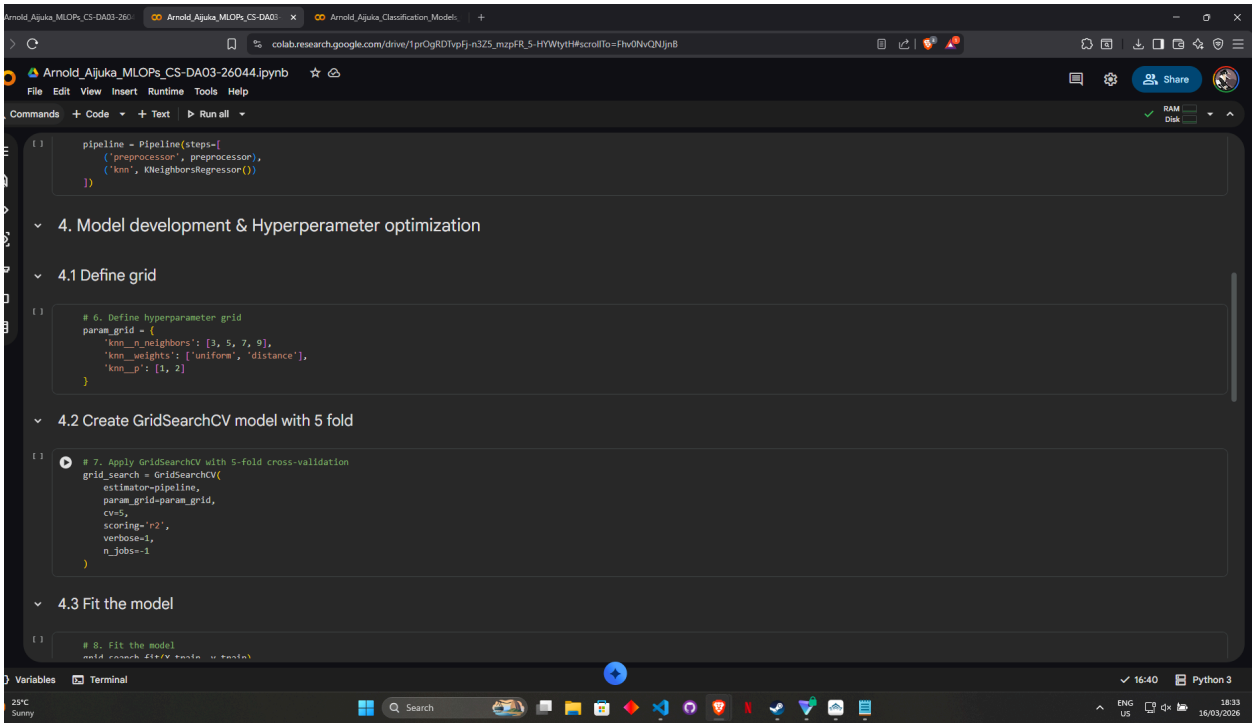
4.1 Define grid

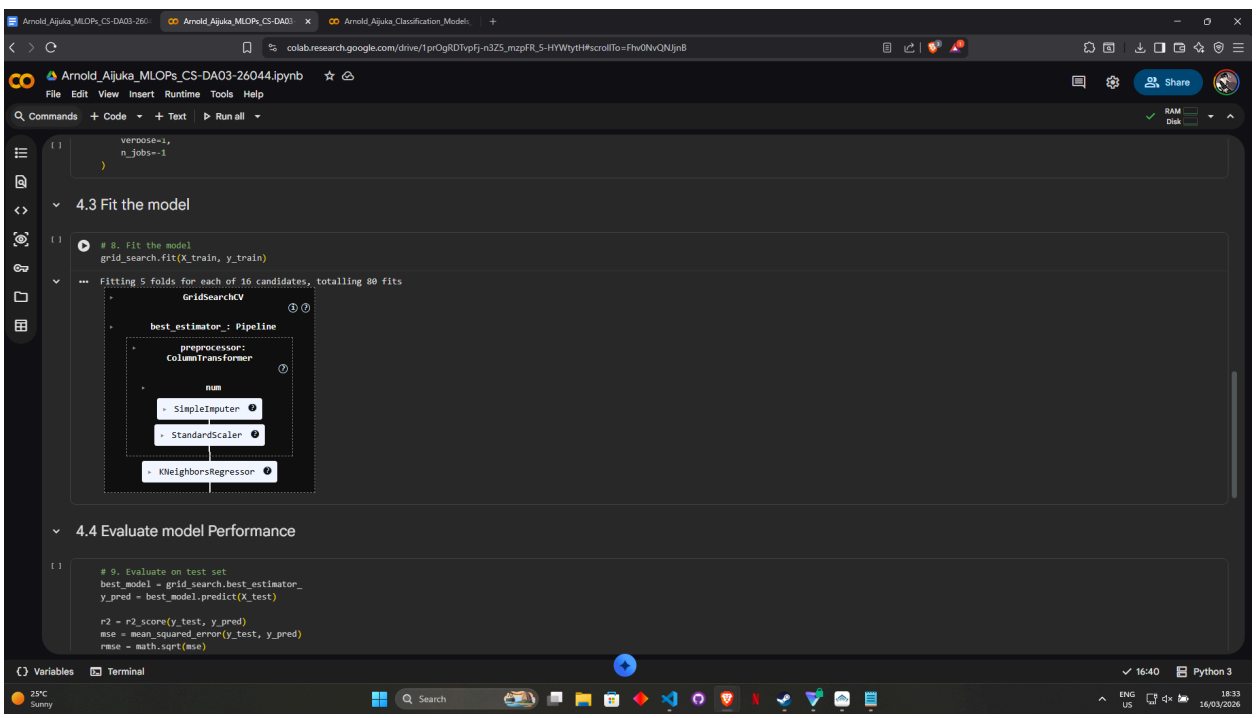
```python
# 6. Define hyperparameter grid
param_grid = {
    'knn__n_neighbors': [3, 5, 7, 9],
    'knn__weights': ['uniform', 'distance'],
    'knn__p': [1, 2]
}
```



Apply GridSearchCV

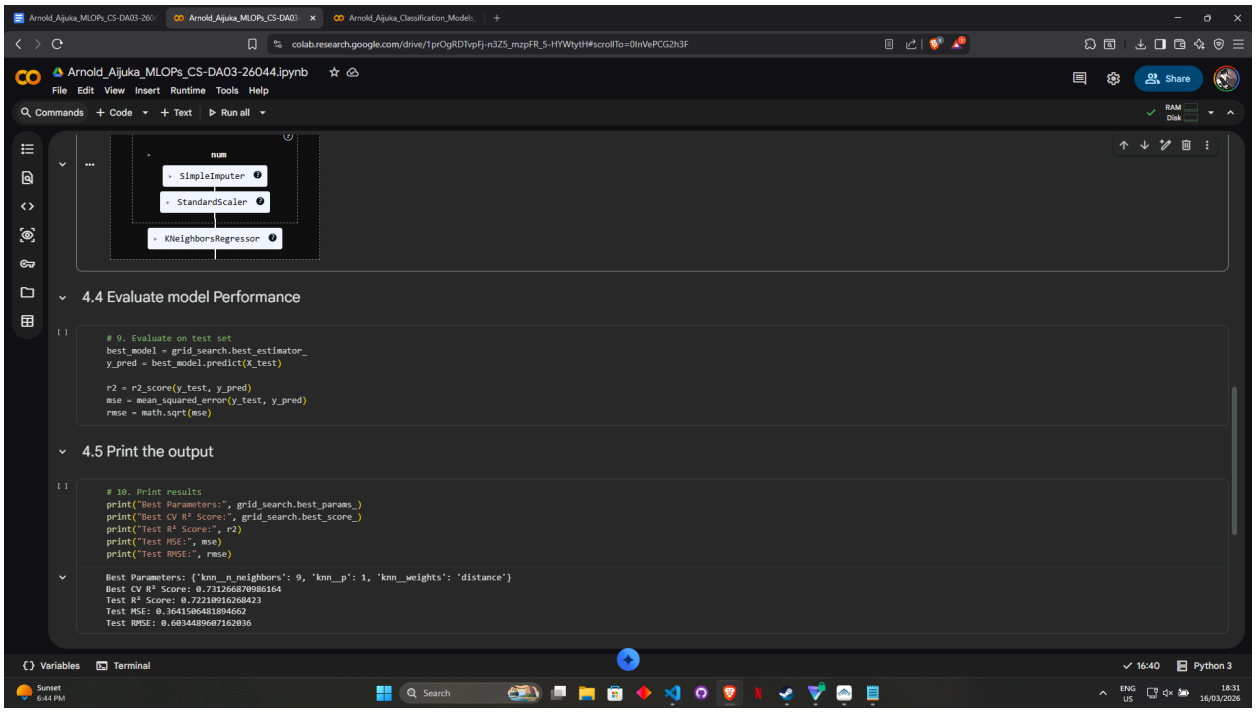
The next step was to create the GridSearchCV model and pass the pipeline (Which has our model and column transfer), the grid parameters, the number of folds that should be done and the scoring method which is r^2.





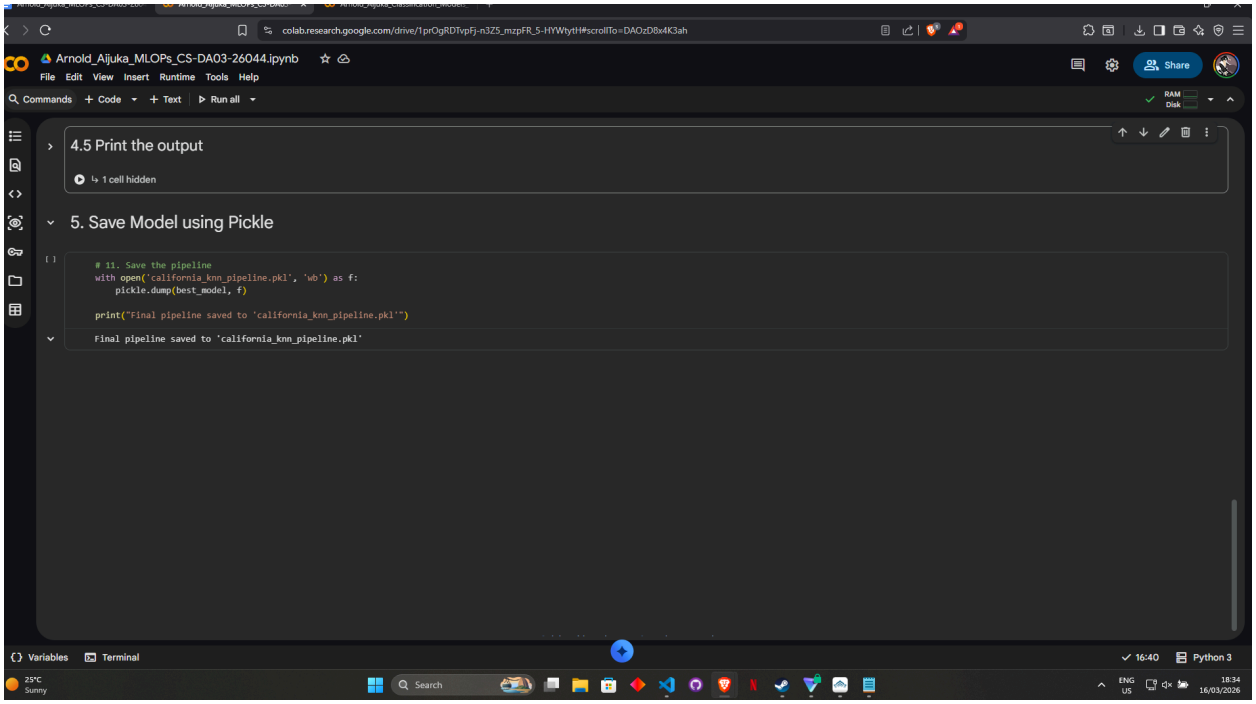
Define an evaluation metric R^2

After tuning the model with **GridSearchCV** which used the $\mathbf{R^2}$ score to find the model that best explains the variance in the data we performed a final evaluation. We included **MSE** and **RMSE** alongside $\mathbf{R^2}$ to get a complete picture: $\mathbf{R^2}$ tells us how well the model understands the relationships between the variables, while **RMSE** tells us exactly how far off our price predictions are on average.



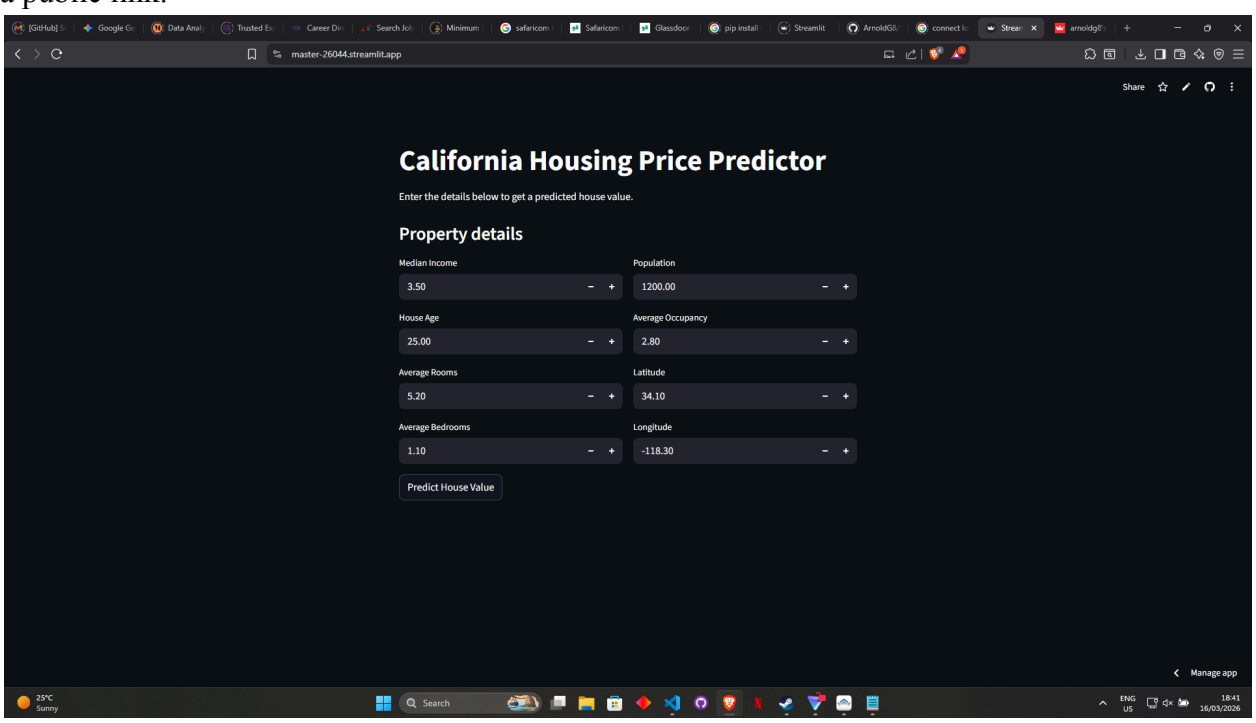
Save the trained pipeline using pickle or joblib.

After developing and evaluating the model we can save it using pickle so that it can be deployed at a later stage.



[OPTIONAL] Using Streamlit to host model for deployment

Rather than utilizing FastAPI or flask, we made use of Streamlit to deploy our ml model through a public link.



Link to Code: ∞ **Arnold_Aijuka_MLOPs_CS-DA03-26044.ipynb**

Link to Streamlit App: **Streamlit KNNRegressor App**

Conclusion

This assignment followings the Machine Learning pipeline from the input stage all the way to the deployment if the optional section is done well, the idea is to show how a model can be built, tested, optimized, locally saved and deployed on a public platform.